

HelixCode Comprehensive Integration Technical Documentation

Deep Analysis, CLI Agent Porting Guide, Integration Plan, Testing Strategy, and
Implementation Roadmap

Version 1.0 | May 2026

Placeholder for table of contents

0

1. Executive Summary

This document provides an exhaustive technical analysis and integration plan for the HelixCode AI CLI agent ecosystem. It covers deep codebase analysis of all 9 primary repositories and 10 supporting submodules, a comparative feature gap analysis against 60+ CLI agent implementations, a comprehensive phased integration plan, and a rigorous testing strategy leveraging Challenges, HelixQA, and LLMsVerifier. The goal is to port every meaningful power feature, innovation, API, workaround, and optimization from all CLI agents into the HelixCode and HelixAgent codebases while ensuring complete test coverage and documentation.

The HelixCode project represents a massive Go-based multi-agent AI coding system with 376 production source files, 258 test files, 87 git submodules, 30+ LLM providers, and 5 specialized agent types. However, critical gaps exist: all 87 git submodules remain uninitialized, key HelixAgent and HelixSpecifier integrations are missing, and several high-value features from CLI agents have only stub implementations. This document addresses every gap with concrete, actionable implementation tasks.

1.1 Key Metrics

Metric	Value
Total Repositories Analyzed	19 (9 primary + 10 supporting)
CLI Agents Analyzed	60+ (10 Tier 1, 9 Tier 2, 41 Tier 3-5)
HelixCode Source Files	376 production + 258 test Go files
HelixAgent Source Files	1,609 Go source files (~1M LOC)
HelixLLM Source Files	100+ Go source files
HelixQA Test Banks	126 test banks (~106K lines)
LLMsVerifier Provider Adapters	25+ adapters, 48 CLI agent generators
Features to Port	50+ unique capabilities identified
Already Implemented	~15 features (30%)
Remaining to Port	~35 features (70%)

2. Repository Inventory and Current Integration Status

This section catalogs all repositories in the Helix ecosystem, their current integration status with HelixCode, and identifies which submodules require initialization, wiring, or new integration paths.

2.1 Primary Repositories

Repository	Go Module	Integration Status	Criticality
HelixCode	dev.helix.code	MAIN PROJECT	CRITICAL
HelixAgent	N/A (not imported)	NOT INTEGRATED	CRITICAL
HelixLLM	N/A (not imported)	NOT INTEGRATED	HIGH
LLMsVerifier	digital.vasic.*	Adapter only, submodule empty	CRITICAL
HelixMemory	N/A (not imported)	NOT INTEGRATED	HIGH
HelixSpecifier	N/A (not imported)	NOT INTEGRATED	MEDIUM
HelixQA	digital.vasic.helixqa	Code exists, submodule empty	CRITICAL
Challenges	digital.vasic.challenges	Code exists, submodule empty	HIGH
Containers	digital.vasic.containers	Code exists, submodule empty	HIGH

2.2 Supporting Submodules

Repository	Purpose	Integration Status
Agentic	Multi-agent orchestration framework	Not integrated
TOON	Task orchestration and optimization network	Not integrated (placeholder)
ToolSchema	Tool schema definitions and validation	Not integrated
VectorDB	Vector database abstraction layer	Not integrated (stubs)
Watcher	File and resource watching system	Not integrated
conversation	Conversation state management	Not integrated (missing dep)
Security	SSRF guard and security utilities	Referenced, submodule empty
Panoptic	Vision and image analysis	Not integrated (pixel-heuristic)
EventBus	Event-driven communication	Not integrated
Concurrency	Concurrency-safe data structures	Integrated via safe.Slice/Store

3. Deep Codebase Analysis

This section provides an in-depth architectural analysis of each primary repository, identifying strengths, weaknesses, implementation status, and integration opportunities.

3.1 HelixCode Architecture

HelixCode is a Go-based multi-agent AI coding system built on a layered architecture. The system provides 8 entry points (CLI, TUI, REST API, WebSocket, Desktop GUI, Aurora OS, Harmony OS, Mobile), a Gin-based HTTP server with 40+ REST endpoints, and a modular internal package structure with 45+ packages. The core architecture follows a layered design: Client Interfaces (CLI/TUI/API/WS) at the top, followed by the Cobra Command Layer, the Gin HTTP Server with route groups, Core Services (Auth, Worker, Task, Project, Session, LLM, Notification, Memory, Workflow), and the Data Layer (PostgreSQL, Redis, Cognee, Memory Providers).

The LLM Provider Layer is the most extensive package, implementing a unified Provider interface supporting 30+ AI providers including 18 cloud providers (OpenAI, Anthropic with prompt caching and extended thinking, Gemini, Azure, Bedrock, VertexAI, Groq, OpenRouter, Qwen, XAI, Copilot, KoboldAI) and 11+ local providers (Ollama, llama.cpp, vLLM, LocalAI, LM Studio, and generic OpenAI-compatible). The ModelManager implements multi-criteria scoring with 60% LLMsVerifier / 40% local heuristic blended scores, hardware-aware model selection, and automatic provider discovery.

3.1.1 Critical Gaps in HelixCode

- ALL 87 git submodules are UNINITIALIZED - build WILL FAIL for any code importing digital.vasic.* packages
- Dual go.mod confusion: root declares Go 1.25.2, HelixCode/ declares Go 1.26, both use module dev.helix.code
- HelixAgent is NOT integrated - no code references, no submodule entry
- HelixSpecifier is NOT integrated - no code references, no submodule entry
- Redis/Memcached memory providers use local maps instead of real connections (stubs)
- Some API handlers return 501 Not Implemented
- Collaboration voting is "first result as consensus" placeholder
- MCP WebSocket origin check returns true for all origins
- Project/workflow routes have no authentication

3.2 HelixAgent Architecture

HelixAgent is a massive Go monorepo with 1,609 Go source files totaling approximately 1 million lines of code, 51 LLM providers, 50+ MCP adapters, 8 application binaries, and 31 constitutional rules governing all development. It incorporates features ported from 60+ CLI agent codebases, organized in the `cli_agents/` directory. The internal structure includes 65+ packages covering agents, tools, features, browser, memory, planning, background services, MCP, containers, challenges, verifier, and much more.

Key already-implemented systems include KAIROS (always-on background assistant with tick-based decision making), Dream System (memory consolidation with 3-gate triggers), Swarm (multi-agent coordination with XML messaging and shared scratchpad), YOLO Classifier (7-rule heuristic auto-approval with history-based risk assessment), EditBlock (Aider-style search/replace editing), RepoMap (basic repository mapping), and Sandbox (Docker/Podman execution).

3.2.1 Stub and Incomplete Implementations

- SubAgent `executeTask()` is a SIMULATION - returns hardcoded results, does not call LLMs
- Tree-sitter NOT integrated - RepoMap uses basic regex, not AST parsing
- Dream phases 2-3 are no-ops - gather and consolidation just log messages
- KAIROS context is STUB - `getGitBranch` and `getModifiedFiles` return empty
- Voice package is EMPTY - no speech recognition implementation
- Seatbelt (macOS sandbox) is a STUB - no actual sandbox-exec integration
- No TUI implementation exists
- No evaluation/benchmark framework
- `loadObservations` in KAIROS is placeholder

3.3 HelixLLM Architecture

HelixLLM implements a sophisticated single-binary mode system with 6 operating modes (full, gateway, brain, knowledge, agents, control) and clean layer separation: Gateway to FallbackChain to Brain to Knowledge to Agents to Control. It supports 10 LLM provider integrations (llama.cpp, OpenAI, Anthropic, Chutes, OpenRouter, HuggingFace, Nvidia, Cerebras, SambaNova, Together), 22+ built-in agent tools, full RAG pipeline with hybrid BM25+semantic search and MMR reranking, and 80+ test files covering unit, integration, e2e, stress, benchmark, security, and chaos testing.

Power features include intelligent multi-provider fallback with LLMsVerifier scoring and circuit breakers, small model optimization (system prompt replacement, tool compression, request orchestration), HTTP/3 + HTTP/2 dual-stack with TLS 1.3, 3-tier agent memory (working/episodic/semantic), and multi-agent coordination with a 4-phase pipeline. However, critical gaps include: PostgreSQL, Kafka, and ClickHouse are configured but not wired, no persistent conversation storage (in-memory only), distributed mode is scaffolding not implemented, and tool call extraction uses fragile string-matching heuristics.

3.4 HelixQA Architecture

HelixQA v0.2.0 is the quality assurance engine with 383 source files, 350 test files, and 126 test banks containing approximately 106K lines of test definitions. Its flagship feature is the autonomous 4-phase LLM-driven QA session (Setup, Doc-Driven, Curiosity, Report). Key capabilities include a dual-model vision architecture, Anti-Bluff system (CONST-035), cognitive memory layer, cheaper vision with 5 adapters, PELT change-point detection, and 13 structured action types. Critical gaps include inability to build standalone (requires sibling dependencies), 4K+ prose actions in test banks that need migration to structured types, Playwright runtime pending, and iOS executor missing.

3.5 LLMsVerifier Architecture

LLMsVerifier v2.0 implements a strategy pattern with circuit breaker, verification pipeline, and scoring engine. Its flagship feature is the mandatory "Do you see my code?" verification with 3-stage testing (meaningful response check, debate prompt, code visibility verification). It provides 48 CLI agent config generators, models.dev integration, (llmsvd) branding suffix, self-improvement feedback loop, and 18+ containerized MCP servers. Critical gaps include 80+ markdown document noise, API key exposure in git history, and 10+ Python scripts that should be Go.

3.6 HelixMemory Architecture

HelixMemory implements an orchestration layer fusing 4 memory backends (Mem0, Cognee, Letta, Graphiti) via a 3-stage fusion engine (collect, deduplicate, rerank). It supports 7 memory types (fact, graph, core, temporal, episodic, procedural, semantic) with auto-classification, 4 full REST API clients with circuit breakers, and production infrastructure with Docker Compose (7 services), PostgreSQL schema (6 tables, 4 views, triggers), and Prometheus metrics. Key gaps include Letta search not passing queries, no write fan-out for redundancy, and trivial consolidation logic.

3.7 HelixSpecifier Architecture

HelixSpecifier implements a 3-pillar fusion engine (SpecKit + Superpowers + GSD) with adaptive ceremony scaling. It provides a 7-phase SDD (Constitution, Specify, Clarify, Plan, Tasks, Analyze, Implement) with phases selected by ceremony level, 10 pre-configured CLI adapters, and 835+ tests across 7 test suites, all race-detector clean. Critical gaps include no persistence (SpecMemory is in-memory only), task execution is stubbed (Superpowers returns hardcoded success), placeholder output without injected DebateFunc, and no spec validation. The highest-value integration opportunity is bridging SpecMemory to HelixMemory for persistent storage and semantic search.

3.8 Challenges and Containers (Special Attention)

The Challenges module is the most architecturally sophisticated supporting module, featuring a constitutional anti-bluff system with no bypass possible, 16 built-in assertion evaluators, a multi-platform userflow automation framework with 21 adapters and 19 challenge templates, topological dependency ordering using Kahn algorithm, liveness monitoring with progress reporters, and direct integration with the Containers module via callback adapters. It contains 209+ tests in the userflow package alone.

The Containers module is the most feature-rich supporting module, supporting 7 container runtime backends (Docker, Podman, Kubernetes, LXD, nerdctl, CRI-O, plus RemoteRuntime over SSH), GPU-aware scheduling with 6 strategies, lifecycle management (lazy boot, idle shutdown, concurrency semaphores), a ctop TUI, remote distribution with failover, and 20 event types with aggressive test coverage and multiple coverage escalation files. Both modules use the CLI-shim pattern (shelling out to CLI tools rather than native SDKs), and the Containers module should be upgraded with native Go SDK clients for Docker and Kubernetes.

4. CLI Agent Comparative Analysis

This section provides a detailed comparative analysis of what each CLI agent does better than the current HelixCode/HelixAgent codebase, identifying missing features, performance advantages, and game-changing innovations that must be ported.

4.1 Tier 1 CLI Agent Feature Matrix

Feature	claude-code	aider	codex	openhands	cline	continue	roo-code	swe-agent
Git Integration	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Browser Automation	No	No	No	Yes	Yes	No	No	No
Sandbox/Security	Yes	No	Yes	Yes	No	No	No	Yes
Voice Commands	No	Yes	No	No	No	No	No	No
Multi-file Edit	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Repo Mapping (tree-sitter)	Yes	Yes	No	Yes	No	Yes	No	Yes
Evaluation Framework	No	No	No	Yes	No	No	No	Yes
Team/Swarm	Yes	No	No	Yes	No	No	No	No

Plan Mode	Yes	No	No	No	No	No	No	No
Auto-commit	No	Yes	Yes	Yes	Yes	No	Yes	No
Context Providers	No	No	No	No	No	Yes	No	No
Agent Modes	No	Yes	No	No	No	No	Yes	No

4.2 Claude Code Source - Deep Analysis (Priority #1)

Claude Code Source is the highest-priority CLI agent for analysis and porting. As the internal reference implementation that inspired KAIROS, Dream, Teams, and YOLO features in HelixAgent, it contains the canonical implementations that HelixAgent has partially ported but with critical stubs and gaps. The key innovations include the KAIROS always-on assistant (HelixAgent port has stub context gathering), the Dream System for memory consolidation (HelixAgent port has no-op gather/consolidation phases), Team Management with XML communication (HelixAgent port works but lacks consensus voting), and the YOLO Classifier (HelixAgent port has 7 heuristic rules but lacks ML-based classification). Additionally, the Plan Mode with verification gates, the 4-layer Permission System, and the EditBlock format are all features where the original implementation has nuances and optimizations not yet captured in the HelixAgent port.

4.2.1 Power Features to Extract from Claude Code

- **Prompt Caching** - Anthropic-style cache control with `cache_control` markers reducing token usage by 40-60%
- **Extended Thinking** - Automatic chain-of-thought activation for complex tasks with budget tokens
- **SubAgent Lifecycle** - Complete task delegation with real LLM calls (not simulation)
- **Memory Consolidation** - 4-phase dream system with actual signal gathering and pattern extraction
- **Context Window Engineering** - Dynamic context selection and compression to maximize relevant information
- **Auto-Approval with Learning** - ML-based YOLO classifier that improves from execution history
- **Git-Native Workflow** - Automatic commit message generation, branch awareness, diff-based reviews
- **Verification Gates** - Multi-step plan mode with verification checkpoints between phases

4.3 Aider - Git-Native Pair Programming

Aider is the most popular open-source AI pair programming tool with 5.7M+ PyPI installs and 15B+ tokens processed weekly. Its key differentiators that HelixAgent/HelixCode must incorporate include tree-sitter based Repository Mapping (`repomap.py`, 867 lines) supporting 100+ programming languages with intelligent file context selection, the EditBlock Format for surgical code modifications

with search/replace blocks and minimal diff generation, Voice-to-Code integration (voice.py, 187 lines) for hands-free coding, and its Git-Native Workflow with automatic commits, diff reviews, and lint/test integration loops. HelixAgent already has a basic RepoMap using regex instead of tree-sitter, and EditBlock is implemented but lacks the flexible matching algorithms from Aider.

4.4 OpenAI Codex - Sandboxed Execution

Codex provides the definitive implementation of sandboxed command execution that HelixCode must adopt. Key features include macOS Seatbelt integration (/usr/bin/sandbox-exec) for OS-level sandboxing, network-disabled execution by default, a multi-level approval system (full autonomy, interactive, approval-required modes), a rich TUI built with ratatui in Rust, and a JSON-RPC Lite protocol for structured communication with thread history management and conversation summaries. The TUI is particularly important as HelixCode currently lacks a proper terminal UI. The approval system maps directly to HelixAgent YOLO Classifier but provides a more sophisticated multi-gate architecture.

4.5 OpenHands - Autonomous SWE

OpenHands achieves a 77.6% SWE-bench score and provides the most complete autonomous software engineering implementation. Its agent system with AgentHub for pluggable agent architectures maps to HelixAgent SubAgent system but provides more sophisticated agent types. The evaluation framework with SWE-bench integration is critical for measuring HelixCode performance objectively. The event-driven Action-Observation loop provides a cleaner execution model than current HelixAgent implementations. Docker-based runtime sandboxing with E2B integration provides the execution isolation that HelixCode needs. The Theory of Mind module and browser automation for headless web navigation are unique features not present in any other agent.

4.6 Cline - Browser Automation

Cline provides the most complete browser automation implementation among CLI agents. Its headless browser integration enables web navigation, screenshot analysis, API documentation reading, and autonomous task execution with self-correction. The multi-provider support, file context management with gitignore-aware operations, and diff-based editing with a minimal change principle are all valuable features. The autonomous task planning with multi-step workflow execution and self-correction on errors is a key capability that maps to HelixAgent SubAgent but with a more robust iteration loop.

4.7 Continue - Universal IDE Support

Continue provides the most extensible context provider system with @file, @url, @docs, and @codebase context sources, tab-based code completion with multi-line suggestions, a slash command action system (/edit, /comment, /doc, /test), and universal IDE support across VS Code, JetBrains, and any LSP-compatible editor. The modular context provider system is the key innovation - it allows dynamic context injection from arbitrary sources, which would significantly improve HelixCode context building. The action framework maps to HelixCode slash commands but provides a more composable architecture.

4.8 Feature Gap Summary

Feature	Source Agent	Current Status in Helix	Porting Priority
Tree-sitter Repository Mapping	Aider	Basic regex-based only	CRITICAL
Sandboxed Execution (Seatbelt)	Codex	Docker/Podman only	CRITICAL
SubAgent Real LLM Calls	Claude Code	Simulation/hardcoded	CRITICAL
Dream System Consolidation	Claude Code	No-op phases 2-3	CRITICAL
Browser Automation (Headless)	Cline/OpenHands	HelixCode has it; HelixAgent partial	HIGH
Evaluation/SWE-bench Framework	OpenHands/SWE-agent	Not implemented	HIGH
Voice Commands	Aider	Empty package	HIGH
Auto-Commit with Message Gen	Aider/Codex	HelixCode has it; HelixAgent basic	HIGH
Context Providers (@file/@url/@docs)	Continue	Mentions parser exists	HIGH
Agent Modes (Code/Architect/Ask)	Roo Code	Not implemented	MEDIUM
TUI (Terminal User Interface)	Codex (ratatui)	Not implemented	MEDIUM
Lint/Test Fix Iteration Loop	Aider	Not implemented	MEDIUM
ML-Based YOLO Classifier	Claude Code	7-rule heuristic only	MEDIUM
JSON-RPC Protocol	Codex	Not implemented	LOW
Event-Driven Action Loop	OpenHands	Not implemented	LOW

5. Git Submodule Integration via SSH

All submodules MUST be integrated using Git via SSH (not HTTPS) as specified. This section provides the step-by-step procedure for initializing, wiring, and verifying all submodules in both HelixCode and HelixAgent projects.

5.1 Submodule Initialization Procedure

The following steps must be executed in order for both the HelixCode and HelixAgent repositories. Each step includes verification commands to ensure proper initialization. Before beginning, ensure SSH keys are configured for both github.com and gitlab.com remotes, and that the SSH config includes entries for both HelixDevelopment and vasic-digital organizations.

5.1.1 Convert Existing HTTPS Submodules to SSH

For each repository, convert all .gitmodules entries from HTTPS to SSH URLs. This is critical because the projects currently reference submodules with git@github.com: SSH URLs in their .gitmodules files, but the clone operations in this environment used HTTPS as a fallback. The canonical SSH format must be maintained going forward.

5.1.2 Add Missing Submodules to HelixCode

HelixCode currently lacks submodule entries for HelixAgent, HelixSpecifier, and HelixMemory. These must be added as proper Git submodules with SSH URLs and fully integrated into the go.mod replace directives. The following submodules need to be added:

- HelixAgent: git@github.com:HelixDevelopment/HelixAgent.git - Path: HelixAgent/
- HelixSpecifier: git@github.com:HelixDevelopment/HelixSpecifier.git - Path: HelixSpecifier/
- HelixMemory: git@github.com:HelixDevelopment/HelixMemory.git - Path: HelixMemory/
- HelixLLM: git@github.com:HelixDevelopment/HelixLLM.git - Path: HelixLLM/

5.1.3 Initialize All Submodules Recursively

Execute the following command sequence to recursively initialize and update all submodules across both repositories. This must be done on a machine with SSH access configured. The recursive flag is essential because HelixAgent, HelixLLM, and Challenges all have their own nested submodules that must also be initialized.

```
git submodule update --init --recursive
```

5.2 go.mod Replace Directives

After adding submodules, the go.mod file must be updated with replace directives for each submodule. The following table shows the required additions for HelixCode:

Module Path	Local Path	Status
digital.vasic.containers	../Containers	Exists, needs init
digital.vasic.helixqa	../HelixQA	Exists, needs init
digital.vasic.challenges	../Challenges	Exists, needs init
digital.vasic.security	../Security	Exists, needs init
dev.helix.agent	../HelixAgent	NEW - must add
dev.helix.specifier	../HelixSpecifier	NEW - must add
dev.helix.memory	../HelixMemory	NEW - must add
dev.helix.llm	../HelixLLM	NEW - must add

6. Comprehensive Phased Integration Plan

This plan is divided into 6 phases spanning 12-16 weeks. Each phase contains fine-grained tasks with specific deliverables, dependencies, and verification criteria. No feature, optimization, or workaround from any CLI agent may be skipped, omitted, or relativized.

6.1 Phase 0: Foundation Repair (Week 1-2)

Before any feature porting can begin, the foundation must be solid. This phase addresses all critical infrastructure issues that currently block development and testing.

Task ID	Task	Deliverable	Priority
0.1	Initialize ALL git submodules recursively via SSH	All 87+ submodules checked out and building	CRITICAL
0.2	Fix dual go.mod - remove root go.mod, keep HelixCode/go.mod	Single go.mod with consistent Go version	CRITICAL
0.3	Add missing submodules (HelixAgent, HelixSpecifier, HelixMemory, HelixLLM)	git submodule add entries with SSH URLs	CRITICAL
0.4	Wire go.mod replace directives for all new submodules	Replace directives for dev.helix.agent, specifier, memory, llm	CRITICAL
0.5	Verify build succeeds with all submodules initialized	go build ./... passes without errors	CRITICAL

0.6	Resolve digital.vasic.messaging missing dependency	Import path resolved or stub created	HIGH
0.7	Clean up 80+ contradictory markdown status files	Archive to docs/archive/, single STATUS.md	MEDIUM
0.8	Secure public project/workflow API routes	JWT auth middleware on all /api/v1/projects/* routes	HIGH

6.2 Phase 1: Critical Feature Porting (Week 3-5)

This phase ports the most critical features from CLI agents that are currently stubs or missing entirely. These features are required for basic production functionality.

Task ID	Task	Source Agent	Deliverable
1.1	Implement tree-sitter Repository Mapping	Aider	internal/tools/repomap/ with AST parsing, 100+ languages
1.2	Implement Seatbelt sandbox for macOS	Codex	internal/tools/sandbox/seatbelt.go with /usr/bin/sandbox-exec
1.3	Wire SubAgent executeTask() to real LLM calls	Claude Code	SubAgent delegates to LLM via provider layer
1.4	Implement Dream gather and consolidation phases	Claude Code	Phase 2: KAIROS log analysis; Phase 3: LLM-based extraction
1.5	Fix KAIROS context gathering (git branch, modified files)	Claude Code	Real git command execution replacing stubs
1.6	Port Aider EditBlock flexible matching algorithms	Aider	Fuzzy matching, whitespace normalization, line-range matching
1.7	Implement JSON-RPC Lite protocol adapter	Codex	internal/protocol/jsonrpc.go for structured communication
1.8	Port OpenHands Action-Observation loop	OpenHands	internal/agents/actionloop.go with event stream

6.3 Phase 2: High-Priority Features (Week 6-8)

This phase ports high-priority features that significantly enhance capabilities but are not blocking for basic operation.

Task ID	Task	Source Agent	Deliverable
---------	------	--------------	-------------

2.1	Port browser automation to HelixAgent	Cline/OpenHands	internal/browser/ with headless Chrome, screenshots, DOM
2.2	Implement SWE-bench evaluation framework	OpenHands/SWE-agent	internal/benchmark/ with SWE-bench runner and metrics
2.3	Implement voice command system	Aider	internal/agents/voice/ with speech recognition and command routing
2.4	Enhance auto-commit with AI message generation	Aider/Codex	Diff analysis + LLM commit message + conventional commits
2.5	Port Continue context provider system	Continue	internal/context/providers/ with @file, @url, @docs, @codebase
2.6	Bridge HelixSpecifier SpecMemory to HelixMemory	HelixSpecifier	Persistent spec storage with semantic search
2.7	Wire HelixLLM as primary LLM provider for HelixCode	HelixLLM	Replace direct provider layer with HelixLLM gateway
2.8	Implement HelixMemory integration in HelixCode	HelixMemory	Memory providers connected to agent memory system
2.9	Port lint/test fix iteration loop	Aider	Automatic lint/test/fix cycle with error analysis and retry
2.10	Add native Go SDK clients for Containers	Containers	Replace CLI-shims with Docker/K8s Go SDK calls

6.4 Phase 3: Medium-Priority Features (Week 9-11)

Task ID	Task	Source Agent	Deliverable
3.1	Implement Agent Modes (Code/Architect/Ask/Debug)	Roo Code	internal/agents/modes/ with mode-specific behavior
3.2	Build TUI with ratatui-style interface	Codex	applications/terminal-ui/ with chat, file browser, commands
3.3	Implement ML-based YOLO classifier	Claude Code	Train model from execution history for auto-approval
3.4	Port OpenHands event-driven architecture	OpenHands	Event bus with action/observation stream processing
3.5	Implement Panoptic ML-based vision	Panoptic	Replace pixel heuristics with ML model for image analysis
3.6	Wire Challenges anti-bluff into HelixCode	Challenges	Constitutional anti-bluff verification for all operations

3.7	Implement conversation persistent storage	HelixLLM	PostgreSQL-backed conversation persistence layer
3.8	Add HelixQA autonomous QA session integration	HelixQA	4-phase LLM-driven QA with Setup/Doc/Curiosity/Report

6.5 Phase 4: UI/WEB/REST/CLI/TUI Coverage (Week 12-13)

This phase ensures all applicative components are fully covered across all interface types: UI (Desktop GUI), WEB (REST API + WebSocket), CLI (Command Line), TUI (Terminal UI), and REST (API endpoints). Every feature ported in Phases 1-3 must be accessible from every interface.

- **REST API:** Add endpoints for all new features (/api/v1/agents/modes, /api/v1/repomap, /api/v1/sandbox, /api/v1/voice, /api/v1/context, /api/v1/benchmark)
- **CLI:** Add Cobra commands for all new features (helix repomap, helix sandbox run, helix voice, helix benchmark, helix mode)
- **TUI:** Integrate all features into the terminal UI with panels for repomap, browser, voice, and mode switching
- **Desktop GUI:** Update Fyne desktop application with new feature panels and controls
- **WebSocket:** Expose all features via MCP protocol for real-time integration
- **Mobile:** Update shared mobile core with API bindings for new features

6.6 Phase 5: Testing and Hardening (Week 14-16)

This phase implements the comprehensive testing strategy detailed in Section 7 and performs security hardening, performance optimization, and documentation finalization. Every implementation from Phases 1-4 must achieve 90%+ test coverage and pass all Challenges and HelixQA test suites.

7. Testing Strategy

The testing strategy leverages three pillars: the Challenges framework for constitutional anti-bluff verification, HelixQA for autonomous QA sessions with LLM-driven test generation, and LLMsVerifier for model quality assurance. No change or implementation is considered complete until it passes all three testing pillars with 90%+ coverage.

7.1 Testing Pyramid

Level	Type	Tool/Framework	Coverage Target
-------	------	----------------	-----------------

L1	Unit Tests	Go testing + testify	90%+ line coverage
L2	Integration Tests	Go testing + Docker Compose	All module interactions
L3	E2E Tests	Challenges scripts + HelixQA	All user workflows
L4	QA Sessions	HelixQA autonomous 4-phase	All features verified
L5	Model Verification	LLMsVerifier scoring	All LLM providers verified
L6	Benchmark Tests	Go benchmark + SWE-bench	Performance baselines
L7	Security Tests	Security module + OWASP	All attack vectors
L8	Chaos Tests	HelixLLM chaos suite	Failure resilience

7.2 Challenges Integration

The Challenges framework provides 16 built-in assertion evaluators, 19 challenge templates, and topological dependency ordering. Every feature implementation must have corresponding challenge scripts that verify the feature works correctly in isolation and in combination with other features. The constitutional anti-bluff system (CONST-035 equivalent) must be applied to ensure no feature claims to work without actual verification. Challenge scripts are organized by phase and must pass before any phase is considered complete.

7.2.1 Required Challenge Scripts by Phase

Phase	Challenge	Verifies
Phase 0	submodule-init-check	All submodules initialized and buildable
Phase 0	go-mod-consistency	Single go.mod, correct replace directives
Phase 1	repomap-tree-sitter	AST parsing produces correct symbol tables
Phase 1	sandbox-seatbelt	Commands run in isolated macOS sandbox
Phase 1	subagent-real-llm	SubAgent makes actual LLM API calls
Phase 1	dream-consolidation	Dream phases produce real memory entries
Phase 2	browser-automation	Headless browser navigates and captures
Phase 2	swe-bench-eval	SWE-bench test cases pass above threshold
Phase 2	context-providers	All @mention types resolve correctly
Phase 3	agent-modes	Each mode produces mode-appropriate output
Phase 3	anti-bluff-verify	No feature returns bluff/unverified results
Phase 4	api-completeness	All features accessible via REST/CLI/TUI/WS

Phase 5	coverage-gate	90%+ test coverage across all packages
---------	---------------	--

7.3 HelixQA Test Suite Integration

HelixQA provides 126 test banks with approximately 106K lines of test definitions and an autonomous 4-phase LLM-driven QA session. Each feature implementation must undergo a HelixQA autonomous session that validates the feature through Setup, Documentation-Driven testing, Curiosity-driven exploration, and comprehensive Report generation. The Anti-Bluff system (CONST-035) ensures that no test can be bypassed and all results are verified against actual execution rather than claimed behavior. Additionally, the Cheaper Vision system with 5 adapters can validate UI components across different rendering contexts.

7.4 LLMsVerifier Quality Gates

LLMsVerifier provides mandatory "Do you see my code?" verification for all LLM interactions. Every feature that involves LLM calls must pass through the 3-stage verification pipeline: (1) meaningful response check, (2) debate prompt to test consistency, and (3) code visibility verification ensuring the LLM has actually seen and understood the codebase. The 48 CLI agent config generators ensure that every supported LLM provider is properly configured, and the scoring engine provides quantitative quality metrics for each provider and model combination. Circuit breakers and fallback models ensure that degraded LLM performance does not result in incorrect feature behavior.

8. API Specifications for New Integrations

This section defines the API interfaces for all new integration points between HelixCode, HelixAgent, and the supporting submodules.

8.1 HelixAgent Integration API

HelixAgent must be integrated as both a Go module dependency and a runtime service. The module integration provides direct access to HelixAgent internal packages (agents, tools, features, mcp) while the service integration enables remote agent execution via gRPC or REST API.

Endpoint/Interface	Method	Purpose
/api/v1/agents/execute	POST	Execute a task via HelixAgent SubAgent
/api/v1/agents/swarm/create	POST	Create a multi-agent swarm
/api/v1/agents/swarm/{id}/assign	POST	Assign task to swarm agent

/api/v1/agents/modes	GET/PUT	Get/set agent mode (code/architect/ask/debug)
/api/v1/agents/dream/trigger	POST	Manually trigger Dream consolidation
/api/v1/agents/kairos/status	GET	Get KAIROS observation and action history
/api/v1/agents/yolo/classify	POST	Classify tool execution risk level
/api/v1/repomap/generate	POST	Generate tree-sitter repository map
/api/v1/sandbox/execute	POST	Execute command in sandboxed environment
/api/v1/voice/command	POST	Process voice command via speech recognition
/api/v1/context/resolve	POST	Resolve @mention context providers
/api/v1/benchmark/run	POST	Run evaluation benchmark (SWE-bench, etc.)

8.2 Submodule Interface Contracts

Each supporting submodule must expose a clean Go interface that HelixCode and HelixAgent can consume without tight coupling. The following table defines the required interface contracts:

Submodule	Primary Interface	Key Methods
Challenges	Verifier	RegisterChallenge(), RunChallenge(), GetResults()
Containers	RuntimeManager	CreateContainer(), ExecuteInContainer(), HealthCheck()
Security	SSRFGuard	ValidateURL(), CheckIPAlternative(), AuditRequest()
Agentic	Orchestrator	Plan(), Execute(), Coordinate()
TOON	TaskNetwork	SubmitTask(), GetStatus(), Optimize()
ToolSchema	SchemaRegistry	Register(), Validate(), GenerateSpec()
VectorDB	VectorStore	Insert(), Search(), Delete(), HybridSearch()
Watcher	FileWatcher	Watch(), OnChange(), Stop()
Conversation	StateManager	CreateSession(), AddMessage(), GetContext()
Panoptic	VisionAnalyzer	AnalyzeImage(), ExtractText(), Classify()

9. Risk Analysis and Mitigation

This section identifies the key risks in the integration effort and provides mitigation strategies for each.

Risk	Impact	Probability	Mitigation
Submodule initialization reveals broken builds	HIGH	HIGH	Phase 0 dedicated to build verification; incremental init
API incompatibilities between submodules	HIGH	MEDIUM	Adapter pattern with versioned interfaces; compatibility shims
Tree-sitter CGO compilation issues	MEDIUM	MEDIUM	Pre-built WASM parsers as fallback; CI matrix testing
LLM provider API changes break integrations	MEDIUM	HIGH	LLMsVerifier circuit breakers; fallback model lists; adapter versioning
Test coverage targets unrealistic for stub code	MEDIUM	LOW	Incremental coverage gates: 60% Phase 1, 75% Phase 3, 90% Phase 5
HelixMemory backends unavailable in test env	LOW	MEDIUM	In-memory providers for testing; Docker Compose for integration
Cross-platform sandbox (Seatbelt/Linux disparities)	MEDIUM	HIGH	Runtime detection with graceful fallback; container-based universal sandbox
Performance regression from added abstraction layers	MEDIUM	MEDIUM	Benchmark gates in CI; profiling at each phase boundary
Voice recognition accuracy insufficient	LOW	MEDIUM	Multiple STT providers with fallback; command confirmation step
SWE-bench evaluation reveals lower than expected scores	HIGH	MEDIUM	Iterative improvement with prompt engineering; multi-agent consensus approaches

10. Implementation Timeline

The following timeline provides a week-by-week breakdown of the 16-week integration effort with milestones, dependencies, and deliverables.

Week	Phase	Key Deliverables	Milestone
1-2	Phase 0	All submodules initialized, build green, go.mod fixed	Foundation Ready
3	Phase 1	Tree-sitter repomap, Seatbelt sandbox	Critical Infra
4	Phase 1	SubAgent real LLM, Dream consolidation	Agent Core
5	Phase 1	KAIROS context, EditBlock matching, JSON-RPC	CLI Agent Core

6	Phase 2	Browser automation, SWE-bench framework	Automation Ready
7	Phase 2	Voice commands, auto-commit, context providers	UX Features
8	Phase 2	HelixMemory bridge, HelixLLM integration, Containers SDK	Module Integration
9	Phase 3	Agent modes, TUI framework, ML YOLO	Advanced Features
10	Phase 3	Event-driven arch, Panoptic vision, anti-bluff	Verification Layer
11	Phase 3	Persistent conversations, HelixQA sessions	QA Integration
12	Phase 4	REST API endpoints for all features	API Complete
13	Phase 4	CLI commands, TUI panels, WebSocket, Desktop, Mobile	UI/UX Complete
14	Phase 5	Unit + integration tests, 90% coverage	Test Coverage
15	Phase 5	Challenges + HelixQA sessions, security hardening	QA Verified
16	Phase 5	Performance optimization, documentation, release	RELEASE

11. CLI Agent Step-by-Step Porting Guide

This section provides detailed step-by-step guides for incorporating each CLI agent features into HelixAgent and HelixCode, starting with claude-code-source as the priority.

11.1 Claude Code Source Porting Guide

Claude Code Source is the canonical implementation for KAIROS, Dream, Teams, YOLO, and Plan Mode features. The porting process involves analyzing the original TypeScript implementation, extracting the core logic and algorithms, reimplementing in Go with HelixAgent architecture patterns (safe.Slice, safe.Store, atomic operations per CONST-029), and writing comprehensive tests using the Challenges and HelixQA frameworks.

11.1.1 KAIROS Full Implementation

Step 1: Replace `getGitBranch()` stub with actual git command execution using `os/exec` to run `git rev-parse --abbrev-ref HEAD`. **Step 2:** Replace `getModifiedFiles()` stub with `git status --porcelain` parsing. **Step 3:** Implement `loadObservations()` to read the last 3 daily log files from the `kairos logs` directory. **Step 4:** Wire the `onDecision` callback to actual LLM calls through the HelixAgent LLM provider layer, providing the `TickPrompt` as context. **Step 5:** Implement proactive observation sources: file system watcher for workspace changes, git hook integration for commit/push events, and process monitor for

development activity. **Step 6:** Add KAIROS REST API endpoints for observation history, action history, and manual tick triggering. **Step 7:** Write Challenge script verifying KAIROS detects and responds to workspace changes.

11.1.2 Dream System Full Implementation

Step 1: Implement gatherPhase to search KAIROS daily logs for patterns, extract session transcripts, and identify drifting memories (accessed but never updated). **Step 2:** Implement consolidationPhase using LLM calls to extract patterns from gathered signals, identify facts worth preserving, update existing memories with new information, translate relative dates to absolute, and remove disproven facts. **Step 3:** Enhance cleanupPhase from simple line trimming to intelligent MEMORY.md maintenance with stale pointer removal and contradiction resolution. **Step 4:** Wire Dream to HelixMemory for persistent storage across restarts. **Step 5:** Add Dream session API endpoints for triggering, monitoring, and reviewing dream sessions. **Step 6:** Write HelixQA autonomous session verifying Dream produces meaningful consolidations.

11.2 Aider Porting Guide

Step 1: Port tree-sitter integration from Aider repomap.py. Create internal/tools/repomap/treesitter.go with CGO bindings to tree-sitter C library. Implement language detection and parser selection for 100+ languages. **Step 2:** Port repository structure analysis - map files to symbols (functions, classes, variables) with signatures, line numbers, and dependencies. **Step 3:** Port intelligent file context selection - rank files by relevance to current task using symbol dependencies and modification recency. **Step 4:** Enhance EditBlock with Aider flexible matching: whitespace normalization, line-range fuzzy matching, and multi-block application with rollback. **Step 5:** Port voice.py integration using Whisper or similar STT service with command routing. **Step 6:** Port lint/test integration loop: run linter, parse errors, feed to LLM for fixes, iterate until clean. **Step 7:** Write Challenges verifying each ported feature against Aider test cases.

11.3 Codex Porting Guide

Step 1: Port Seatbelt sandbox: create internal/tools/sandbox/seatbelt.go that generates Seatbelt policy profiles and executes commands via /usr/bin/sandbox-exec. **Step 2:** Port multi-level approval system: implement ApprovalPolicy with full-autonomy, interactive, and approval-required modes. **Step 3:** Port TUI: create applications/terminal-ui/ with ratatui-style interface using tview, including chat panel, file browser, command palette, and diff viewer. **Step 4:** Port JSON-RPC Lite protocol: implement internal/protocol/jsonrpc.go with structured request/response, thread history, and conversation summaries. **Step 5:** Write Challenges for sandbox isolation verification and approval gate testing.

11.4 OpenHands Porting Guide

Step 1: Port AgentHub pluggable agent architecture: refactor HelixAgent SubAgent to support dynamic agent registration and selection. **Step 2:** Port evaluation framework: create internal/benchmark/ with SWE-bench runner, custom metrics, and result aggregation. **Step 3:** Port event-driven Action-Observation loop: implement internal/agents/actionloop.go with event stream processing, action dispatch, and observation collection. **Step 4:** Port runtime sandboxing with Docker-based execution and E2B integration. **Step 5:** Write Challenges with SWE-bench test cases and performance thresholds.

11.5 Cline and Continue Porting Guide

Cline Porting: Port headless browser integration to HelixAgent internal/browser/ with Chrome DevTools Protocol. Implement autonomous task planning with multi-step execution and self-correction. Port diff-based editing with minimal change principle. **Continue Porting:** Port modular context provider system with @file, @url, @docs, @codebase resolvers. Port slash command action system with composable commands. Port tab-based code completion with multi-line suggestions. Both require Challenge scripts for browser navigation verification and context resolution correctness.

12. Cross-Module Dependency Graph

The following table maps the dependencies between all modules and identifies the integration order required to satisfy all dependency chains. Modules must be integrated in topological order to prevent compilation failures.

Module	Depends On	Required By	Integration Order
Concurrency	(none)	HelixAgent, HelixLLM, all	1
Security	(none)	HelixCode, HelixAgent	1
Containers	(none)	HelixCode, HelixAgent	2
Challenges	Containers	HelixCode, HelixQA	3
EventBus	(none)	HelixAgent, HelixLLM	1
VectorDB	(none)	HelixMemory, HelixLLM	2
ToolSchema	(none)	HelixAgent, HelixLLM	2
Agentic	EventBus, Concurrency	HelixAgent, HelixLLM	3
TOON	Agentic, Concurrency	HelixLLM	4

Watcher	Concurrency	HelixAgent	3
Conversation	EventBus, Messaging	HelixAgent, HelixLLM	4
Panoptic	(none)	HelixCode	3
LLMsVerifier	(none)	HelixCode, HelixLLM, HelixQA	2
HelixMemory	VectorDB, Concurrency	HelixCode, HelixAgent	4
HelixLLM	Agentic, TOON, VectorDB, Conversation	HelixCode, HelixAgent	5
HelixQA	Challenges, LLMsVerifier	HelixCode	5
HelixSpecifier	HelixMemory, Concurrency	HelixCode, HelixAgent	5
HelixAgent	All above	HelixCode	6

13. Conclusion and Next Actions

This document provides the comprehensive technical foundation for integrating the entire Helix ecosystem into a unified, fully-tested, production-ready AI CLI agent system. The analysis covers 19 repositories totaling millions of lines of code, identifies 50+ features to port from 60+ CLI agents, and provides a detailed 16-week phased implementation plan with 50+ specific tasks, each with deliverables and verification criteria.

The immediate next actions are: (1) Execute Phase 0 on a machine with SSH access to initialize all git submodules recursively, (2) Fix the dual go.mod issue and add missing submodule entries, (3) Verify the complete build succeeds, and (4) Begin Phase 1 with tree-sitter repository mapping and SubAgent real LLM execution. Every feature, optimization, workaround, and innovation identified in this analysis must be ported without exception, and every implementation must pass Challenges, HelixQA, and LLMsVerifier verification before being considered complete.

The testing strategy ensures no gaps, bluffing, or incomplete implementations through three independent verification pillars: Challenges for constitutional anti-bluff checks, HelixQA for autonomous LLM-driven QA sessions, and LLMsVerifier for model quality assurance. The 90% coverage gate at Phase 5 completion ensures production readiness. All UI, WEB, REST, CLI, TUI, and other applicative components must be fully covered across all interface types by Phase 4, ensuring no feature is inaccessible from any supported interface.